Project - High Level Design

On

Multimodal Manufacturing Creator

Course Name: Gen AI

Institution Name: Medicaps University - Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Om Tiwari	EN22CS301669
02	Paridhi Shirwalkar	EN22CS301684
03	Nitesh Chourasiya	EN22CS301660
04	Mradul Jain	EN22CS301616

Group Name: 01D6

Project Number: GAI-12

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

Table of Contents

1. Introduction.

    1.1. Scope of the document.
    1.2. Intended Audience
    1.3. System overview.

2. System Design.

    2.1. Application Design
    2.2. Process Flow.
    2.3. Information Flow.
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue.

3. Interfaces

4. State and Session Management

5. Caching

6. Functional Requirements

    6.1. User Authentication & Authorization
    6.2. Infrastructure Configuration Management
    6.3. Provisioning Control & Monitoring
    6.4. Dockerfile (Application Containerization)
    6.5. GitHub Actions (Workflow Configuration)
    6.6. CI/CD Strategy (Continuous Integration & Deployment)

7. Non-Functional Requirements

8. References

9. GitHub

1. Introduction

The Multimodal Manufacturing Creator is a Generative AI application that translates a single text prompt describing a manufacturing concept into two simultaneous outputs: a structured technical narrative produced by a Large Language Model, and a high-fidelity visual prototype generated by a diffusion-based image model. The system demonstrates the synergy between text-to-text and text-to-image AI models working from a shared input to produce richer, multi-dimensional content than either model could produce alone.

This High Level Design document describes the overall architecture of the system, its major components, data flows, interfaces, state management, and both functional and non-functional requirements.

1.1 Scope of the Document

This document covers the high-level architecture and design of the Multimodal Manufacturing Creator application. It describes how the Streamlit frontend, the LLM orchestration layer, and the image generation pipeline interact to deliver multimodal outputs. It does not cover low-level implementation details, deployment infrastructure, or CI/CD pipelines.

1.2 Intended Audience

This document is intended for the following stakeholders:

- Datagami course evaluators and industry mentors assessing the GenAI project
- Developers who may extend the system with additional modalities or features
- Technical reviewers evaluating the API integration and architectural decisions
- Academic peers seeking to understand multimodal AI application design at a high level

1.3 System Overview

The system accepts a natural language manufacturing concept from the user through a Streamlit web interface. Upon submission, two independent API calls are made from the same input string. The first invokes Google Gemini 2.5 Flash via the google-generativeai Python SDK to generate a five-section technical narrative. The second constructs an HTTP GET request to the Pollinations.ai Flux diffusion model endpoint to generate an engineering illustration. Both outputs are rendered side by side in the Streamlit interface. The application requires no backend server, no database, and maintains no persistent state.

2. System Design

2.1 Application Design

The application is designed as a single-tier stateless web application. The Streamlit framework serves as both the presentation layer and the orchestration layer. All logic — input handling, API invocation, response processing, and output rendering — lives within a single app.py file. This flat architecture is intentional: it keeps the system simple, portable, and focused entirely on demonstrating the multimodal AI workflow rather than infrastructure complexity.

The diagram below illustrates the high-level architecture of the system:

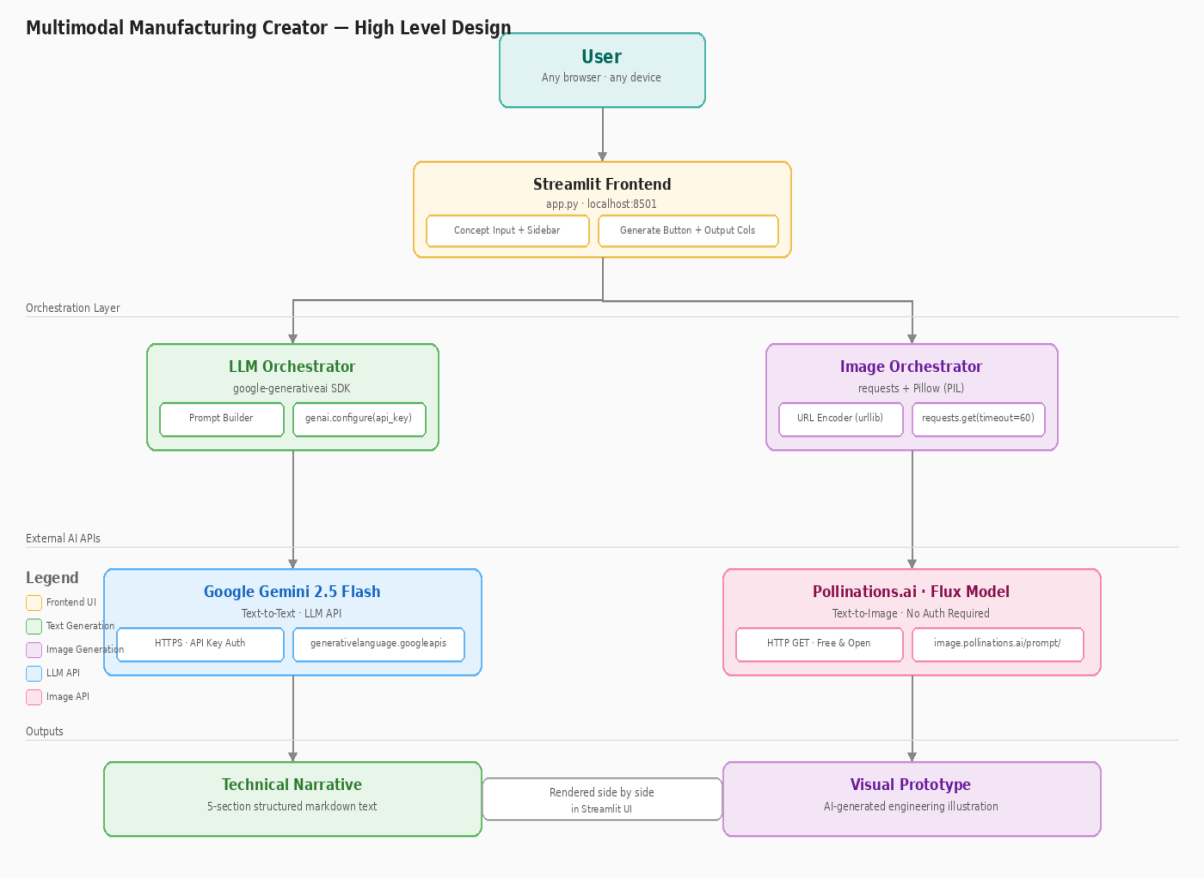

Figure 1: Multimodal Manufacturing Creator - High Level Design

The four logical layers are: the Streamlit UI layer for all user interaction and output rendering; the Orchestration layer in Python that coordinates both API calls from one input; the External API layer comprising Gemini for text and Pollinations for images; and the Output layer where both model responses are decoded and presented together.

2.2 Process Flow

The end-to-end process flow from user input to rendered output is as follows:

Step	Action	Description
1	Concept Entry	User types a manufacturing concept in the text field or selects from the example dropdown. The sidebar captures the Gemini API key and visual style selection.
2	Validation	The app checks that the API key field is non-empty. The Generate button remains disabled until a key is present, preventing unnecessary API calls.
3	Generate Triggered	User clicks Generate. The concept string and selected style are captured and passed to both orchestrators simultaneously.
4	LLM API Call	Gemini 2.5 Flash is invoked with a structured five-section prompt. The SDK handles authentication using the user-provided key.
5	Image API Call	The concept and style strings are concatenated, URL-encoded, and embedded into a Pollinations.ai GET request. A 60-second timeout is applied.
6	Response Handling	The Gemini text response is extracted and rendered as markdown. The image bytes are decoded via Pillow and passed to st.image for display.
7	Output Display	Both outputs appear side by side in two Streamlit columns. The workflow diagram below the outputs explains the pipeline to the viewer.

2.3 Information Flow

Text Generation Path:

- User concept string is embedded into a structured prompt template requesting Overview, Working Principle, Key Components, Manufacturing Application, and Advantages
- Prompt is transmitted to the Gemini API endpoint over HTTPS using the google-generativeai SDK
- API returns a response object whose .text field contains the generated markdown narrative

- The markdown text is rendered directly in the left Streamlit column using st.markdown()

Image Generation Path:

- User concept string and selected visual style are concatenated into a single image prompt string
- The string is URL-encoded using urllib.parse.quote() to produce a safe URL parameter
- An HTTP GET request is sent to the Pollinations.ai endpoint with the encoded prompt, dimensions, model name, and seed as query parameters
- The response body contains raw image bytes which are read into a BytesIO buffer, decoded by PIL.Image.open(), and rendered via st.image() in the right column

No data is written to disk, logged to any service, or retained beyond the active Streamlit session. The Gemini API key never leaves the session memory.

2.4 Components Design

Component	Technology	Responsibility
Streamlit UI	Streamlit (Python)	Renders sidebar settings, concept input, example dropdown, Generate button, two output columns, and workflow diagram
Input Handler	Python	Captures concept text from either the text field or dropdown and passes it to both orchestrators on button click
LLM Orchestrator	google-generativeai	Configures Gemini client with API key, constructs structured prompt, invokes model, extracts and returns response text
Image Orchestrator	requests + Pillow	Encodes prompt to URL, sends HTTP GET to Pollinations.ai, decodes response bytes into PIL image object for display
Error Handler	Python try-except	Independent try-except blocks around each API call ensure one failure never prevents the other output from rendering

Workflow Visualizer	Streamlit native	Static Streamlit columns and info boxes rendering the multimodal pipeline diagram always visible below outputs

2.5 Key Design Considerations

Stateless Architecture

The application holds no server-side state. Each user session is fully independent. There is no database, no file system writes, and no shared memory between sessions. This makes the app inherently scalable and trivially portable.

No Second API Key

Pollinations.ai provides free, unauthenticated access to the Flux diffusion model. The image prompt is encoded directly into the request URL, requiring zero additional credentials from the user. This simplifies onboarding and keeps the demo accessible to anyone with only a Gemini key.

Independent Failure Handling

Both API calls are wrapped in separate try-except blocks. A timeout or quota error on the Gemini side does not prevent the image from being fetched, and vice versa. Each column displays its own error message independently, keeping the other output fully functional.

Prompt Engineering

The LLM prompt is structured to always return exactly five labelled sections regardless of the concept entered. This produces consistent, professionally formatted output suitable for presentations. The image prompt appends style keywords and domain terms to improve visual quality and relevance.

2.6 API Catalogue

API	Type	Auth	Endpoint
Google Gemini 2.5 Flash	REST / SDK	API Key	generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent
Pollinations.ai Flux	HTTP GET	None	image.pollinations.ai/prompt/{encoded}?width=800&height=600&model=flux&nologo=true&seed=42

3. Interfaces

The system exposes one user-facing interface and consumes two external API interfaces.

User Interface

The Streamlit web application is the sole user interface. It is accessible via any browser at localhost:8501 when run locally. The interface consists of a sidebar for configuration and a main area divided into input controls at the top, two output columns in the middle, and a workflow diagram at the bottom. The UI is fully responsive within the Streamlit layout system.

Gemini API Interface

The application communicates with the Google Gemini API via the official google-generativeai Python SDK. The interface is authenticated using a user-provided API key passed at runtime. The request payload is a plain text prompt string. The response is a GenerateContentResponse object from which the .text attribute is extracted.

Pollinations.ai Interface

The image generation interface is a simple HTTP GET request. The entire input is encoded into the URL path and query parameters. No request body, no authentication headers, and no SDK are required. The response is a binary image stream returned with content-type image/jpeg or image/png.

4. State and Session Management

The application is fully stateless by design. Streamlit re-executes the entire app.py script on every user interaction. No data is persisted between runs.

Within a single session the following values are held in Streamlit widget state:

- **Gemini API key-** held in the text_input widget state for the duration of the browser session
- **Selected visual style-** held in the selectbox widget state
- **Current concept text-** held in the text_input or selectbox widget state
- **Generated text response-** lives in the Python variable scope during the active script run
- **Generated image object-** lives in the Python variable scope during the active script run

When the browser tab is closed or the Streamlit server is restarted, all session state is lost. The API key is never written to disk, environment variables, or any external store. There is no login system, no user accounts, and no cross-session data sharing.

5. Caching

The current implementation does not use explicit caching. Each click of the Generate button triggers fresh API calls to both Gemini and Pollinations regardless of whether the same concept was entered previously. This is intentional for a demonstration application it ensures the user always sees a live API call being made.

The Pollinations.ai endpoint uses a fixed seed parameter (seed=42) in the image URL. This means that for the same concept and style combination the Pollinations CDN may return a cached image from their infrastructure, reducing latency on repeated identical requests without any explicit caching logic on the application side.

If the application were extended for production use, st.cache_data decorators could be applied to the LLM and image fetch functions to cache responses by concept string and style, significantly reducing API call volume and response time for repeated queries.

6. Functional Requirements

6.1 User Input and Concept Entry

The system must provide a free-text input field for the user to enter any manufacturing concept. It must also provide a dropdown menu pre-loaded with at least five example concepts that auto-fill the text input on selection. The Generate button must remain disabled until a non-empty API key is detected in the sidebar.

6.2 API Key Configuration

The system must provide a password-type text input in the sidebar for the user to enter their Google Gemini API key at runtime. The key must be used only for the current session and must not be stored, logged, or transmitted to any service other than the Google Gemini API. No default or hardcoded key may be present in the source code.

6.3 Text Generation

Upon triggering generation, the system must invoke the Google Gemini 2.5 Flash model with a structured prompt and return a response containing exactly five labelled sections: Overview, Working Principle, Key Components, Manufacturing Application, and Advantages. The response must be rendered as formatted markdown in the left output column.

6.4 Image Generation

Upon triggering generation, the system must construct a valid Pollinations.ai request URL by URL-encoding the concept and style strings, send an HTTP GET request with a minimum 60-second timeout, decode the binary response into a displayable image, and render it in the right output column. No API key must be required for this call.

6.5 Visual Style Selection

The system must provide at least four visual style options for the image generation: Technical blueprint, Photorealistic industrial, 3D render product prototype, and Schematic exploded view. The selected style must be appended to the image prompt sent to the Pollinations.ai endpoint.

6.6 Error Handling

The system must handle failures in either API call independently. If the Gemini call fails, an error message must be displayed in the left column without affecting the image generation. If the image call fails, an error message must be displayed in the right column without affecting the text generation. In both cases the error message must include the exception detail.

7. Non-Functional Requirements

Category	Requirement	Detail
Performance	Response Time	Text narrative must render within 10 seconds under normal Gemini API load. Image must render within 60 seconds accounting for Pollinations generation time.
Security	API Key Safety	Gemini API key must be masked in the UI input field, held only in session memory, and never appear in logs, source code, or network calls to third parties.
Usability	Zero Setup	A user must be able to run the application with only pip install and streamlit run. No environment variable setup, no config files, and no separate image API key must be required.
Reliability	Independent Outputs	Failure of one API call must never prevent the other output from being displayed. Each output must succeed or fail independently.
Portability	Cross-platform	The application must run on Windows, macOS, and Linux with no platform-specific dependencies beyond the packages listed in requirements.txt.
Maintainability	Single File	All application logic must reside in a single app.py file to maximise readability and minimise onboarding time for new contributors.

8. References

- **Google Gemini API Documentation-** https://ai.google.dev/gemini-api/docs
- **google-generativeai Python SDK-** https://pypi.org/project/google-generativeai
- **Pollinations.ai-** https://pollinations.ai
- **Flux Image Model by Black Forest Labs-** https://blackforestlabs.ai
- **Streamlit Documentation-** https://docs.streamlit.io
- **Pillow (PIL) Library-** https://pillow.readthedocs.io
- **Python urllib.parse-** https://docs.python.org/3/library/urllib.parse.html
- **requests Library-** https://requests.readthedocs.io

9. GitHub

The complete source code for the Multimodal Manufacturing Creator is publicly available at:

https://github.com/omtiwari17/Multimodal-Manufacturing-Creator

File	Description
app.py	Complete Streamlit application containing all UI, LLM orchestration, image generation logic, error handling, and workflow diagram
requirements.txt	Python package dependencies: streamlit, google-generativeai, requests, Pillow
README.md	Project overview, setup instructions, example concepts, tech stack, workflow diagram, and group member details
.gitignore	Excludes Python cache directories, .env files, and Streamlit secrets from version control